

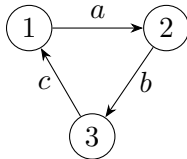
CSCI-UA 102 — Quiz 11

Graphs — Directed Line Graph

Name: _____ NetID: _____

Given a directed graph G , its **directed line graph** $L(G)$ has one vertex for each edge of G . For two edges $e_1 = (a, b)$ and $e_2 = (c, d)$ in G , we add a directed edge from e_1 to e_2 in $L(G)$ if and only if $b = c$ — that is, whenever e_1 ends where e_2 begins.

(1 point) Draw $L(G)$ for the directed graph G below.



Your $L(G)$:

(3 points) Write a method `static <V, E> Graph<E, V> directedLineGraph(Graph<V, E> graph)` that returns the directed line graph of the input. *API reminder:*

```
interface Graph<V, E> {
    PositionList<Vertex<V>>    vertices();
    PositionList<Edge<E, V>>   edges();
    PositionList<Edge<E, V>>   incomingEdges(Vertex<V> v);
    PositionList<Edge<E, V>>   outgoingEdges(Vertex<V> v);
    void    insertVertex(V x);
    void    insertEdge(Vertex<V> from, Vertex<V> to, E x);
}

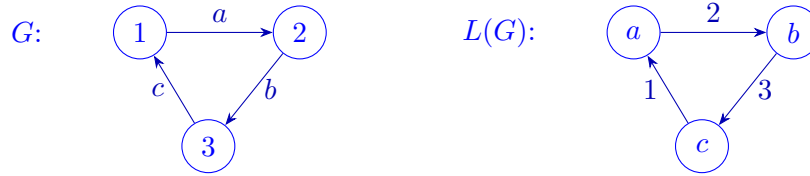
interface Vertex<V> { V getElement(); }
interface Edge<E, V> { E getElement(); Vertex<V>[] getEndpoints(); } // [0]=from, [1]=to
```

Tip: since `insertVertex` returns `void`, grab the newly-added vertex via `L.vertices().last().getElement()`.

(1 point) What is the worst-case asymptotic computational complexity of your implementation?

Solution

Part 1: drawing $L(G)$.



Check each edge: a ends at 2, only b starts at 2 $\Rightarrow a \rightarrow b$ (shared = 2). Similarly $b \rightarrow c$ (shared = 3) and $c \rightarrow a$ (shared = 1). $L(G)$ is also a 3-cycle.

Part 2: Java implementation.

```
static <V, E> Graph<E, V> directedLineGraph(Graph<V, E> graph) {
    Graph<E, V> L = new EdgeListGraph<E, V>();
    Map<Edge<E, V>, Vertex<E>> edgeToVertex = new HashMapSC<>();

    // 1. one vertex in L for each edge in graph, carrying that edge's element
    for (Edge<E, V> e : graph.edges()) {
        L.insertVertex(e.getElement());
        edgeToVertex.put(e, L.vertices().last().getElement());
    }

    // 2. for each vertex v, pair its incoming edges with its outgoing edges
    //     -- each such pair is one edge in L, carrying v's element.
    for (Vertex<V> v : graph.vertices()) {
        for (Edge<E, V> e1 : graph.incomingEdges(v)) {
            for (Edge<E, V> e2 : graph.outgoingEdges(v)) {
                L.insertEdge(edgeToVertex.get(e1),
                             edgeToVertex.get(e2),
                             v.getElement());
            }
        }
    }
    return L;
}
```

Why the map? `insertVertex` returns `void`, so after inserting we grab the just-added vertex with `L.vertices().last()` and stash it in a map keyed by the original edge. Later, when connecting two new vertices, we look them up by the corresponding pair of original edges.

Part 3: complexity.

Let $n = |V(G)|$ and $m = |E(G)|$. Step 1 runs in $O(m)$.

Step 2 visits each vertex v and pairs $\text{indeg}(v)$ incoming edges with $\text{outdeg}(v)$ outgoing edges. In `EdgeListGraph`, each call to `incomingEdges(v)` or `outgoingEdges(v)` scans the whole edge list, costing $O(m)$. So the outer vertex loop contributes $O(nm)$, plus $O(|E(L)|)$ inserts where $|E(L)| = \sum_v \text{indeg}(v) \cdot \text{outdeg}(v) \leq m^2$.

Total: $O(nm + |E(L)|)$ — worst case $O(nm)$ in this representation.

Compared to the naive $O(m^2)$ approach (comparing every pair of edges), this is better whenever $n < m$, which is the usual regime.